

Lab Exercise 4: Basic Blog API with ASP.NET Core 6

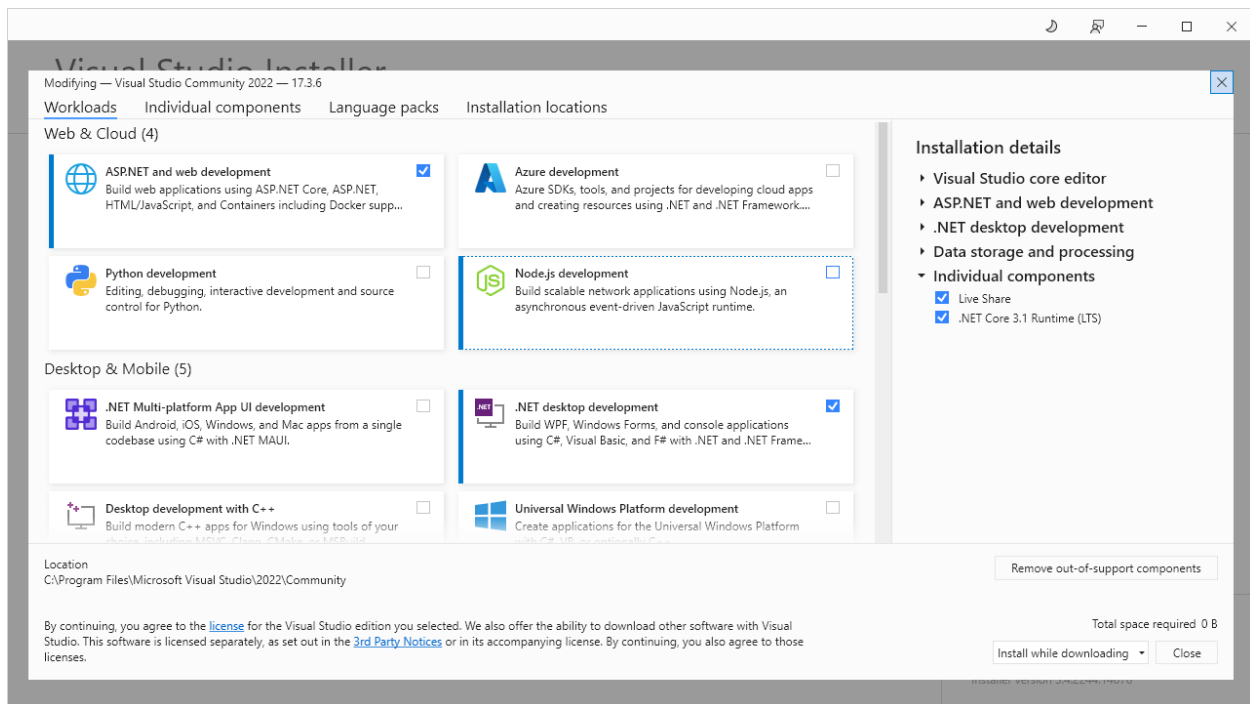
For exercises 2-4, create a blogging site using ASP.NET Web API and SQL Server Database. The site must have log in, registration, and post functionality (adding a post, listing all posts, and showing details of one post).

The web API for the blogging site is made in Lab Exercise 3 using ASP.NET Core 6. It will utilize the code from the previous lab exercise through dependency injection.

The following should be downloaded:

- Postman
- ASP.NET and Web Development

In Visual Studio, go to Tools > Get Tools and Features. Check ASP.NET and web development, and download.



Downloads per project

After setting up, right click each project and click Manage Nuget Packages. Browse for the following packages, ensuring that the latest version for 6.0 is installed:

BlogAPI

- Microsoft.AspNetCore.Authentication.JwtBearer
- Microsoft.IdentityModel.Tokens
- Swashbuckle.AspNetCore
- System.IdentityModel.Tokens.Jwt

Setting up the project

1. Open your solution from lab exercise 2.
2. In the Solution root (named by <lastname>LE2), right click > add > new project.
3. Search for ASP.NET Web API
4. Name it as BlogAPI and choose Next.
5. Choose the .NET 6.0 Framework.
6. Choose None for Authentication type.
7. Check the following:
 - Configure for HTTPS
 - Use controllers
 - Enable OpenAPI support
 - Do not use top-level statements
8. After the project has been created, right click > set as startup project.
9. Right click the Dependencies item of the BlogAPI project, and choose Add Project References
10. Check BlogDataLibrary.
11. You may delete WeatherForecast.cs, and the WeatherForecastController.cs under the Controllers folder.
12. In appsettings.json file add the following information:



```
"Jwt": {
  "Key": "<lastname><birthdate><givenname>",
  "Issuer": "http://localhost:35685/",
  "Audience": "http://localhost:35685/"
},
"ConnectionStrings": {
  "SqlDb": "<connstring>"
}
```

For issuer and audience, go to Project properties --> Debug --> General --> Open Debug Launch Profile UI to get the URL. For connection string, refer to previous exercise on how to obtain it.

13. In Program.cs, add the following code below `builder.services.AddSwaggerGen();`

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(options
=> {
    options.TokenValidationParameters = new TokenValidationParameters
```

```

{
    ValidateIssuer = true,
    ValidateAudience = true,
    ValidateLifetime = true,
    ValidateIssuerSigningKey = true,
    ValidIssuer = builder.Configuration["Jwt:Issuer"],
    ValidAudience = builder.Configuration["Jwt:Audience"],
    IssuerSigningKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]))
};
});

```

The following code uses Session Tokens for authentication.

14. Below the code above, add the following:

```

builder.Services.AddTransient<ISqlData, SqlData>();
builder.Services.AddTransient<ISqlDataAccess, SqlDataAccess>();
builder.Services.AddSingleton<IConfiguration>(builder.Configuration);

```

15. Above the code `app.UseAuthorization()`, add `app.UseAuthentication()`;

App feature: Login

1. Under the Controllers folder, add > controller.
2. Choose the empty API controller.
3. Name it as LoginController.
4. Type in the following code:

```

private IConfiguration _config;
private ISqlData _db;

public LoginController(IConfiguration config, ISqlData db)
{
    _config = config;
    _db = db;
}

```

5. Add a new method inside the program and call it as `GenerateToken()`.

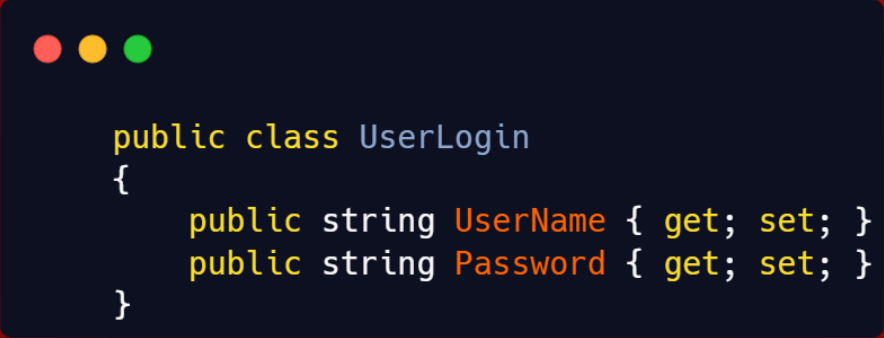
```

        private string GenerateToken(UserModel user)
    {
        var securityKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_config["Jwt:Key"]));
        var credentials = new SigningCredentials(securityKey, SecurityAlgorithms.HmacSha256);
        string userIdStr = user.Id.ToString();
        var claims = new[]
        {
            new Claim(ClaimTypes.NameIdentifier, userIdStr),
        };
        var token = new JwtSecurityToken(_config["Jwt:Issuer"],
            _config["Jwt:Audience"],
            claims,
            expires: DateTime.Now.AddMinutes(15),
            signingCredentials: credentials);

        return new JwtSecurityTokenHandler().WriteToken(token);
    }

```

6. In the BlogDataLibrary Project, add a new class under Models named UserLogin.
7. Set two properties for the class:



```

public class UserLogin
{
    public string Username { get; set; }
    public string Password { get; set; }
}

```

8. Add in the following code to allow the user to login:



```
[AllowAnonymous]
[HttpPost]
[Route("/api/[controller]/login")]
public ActionResult Login([FromBody] UserLogin login)
{
    UserModel user = _db.Authenticate(login.UserName, login.Password);

    if (user != null)
    {
        var token = GenerateToken(user);
        return Ok(token);
    }

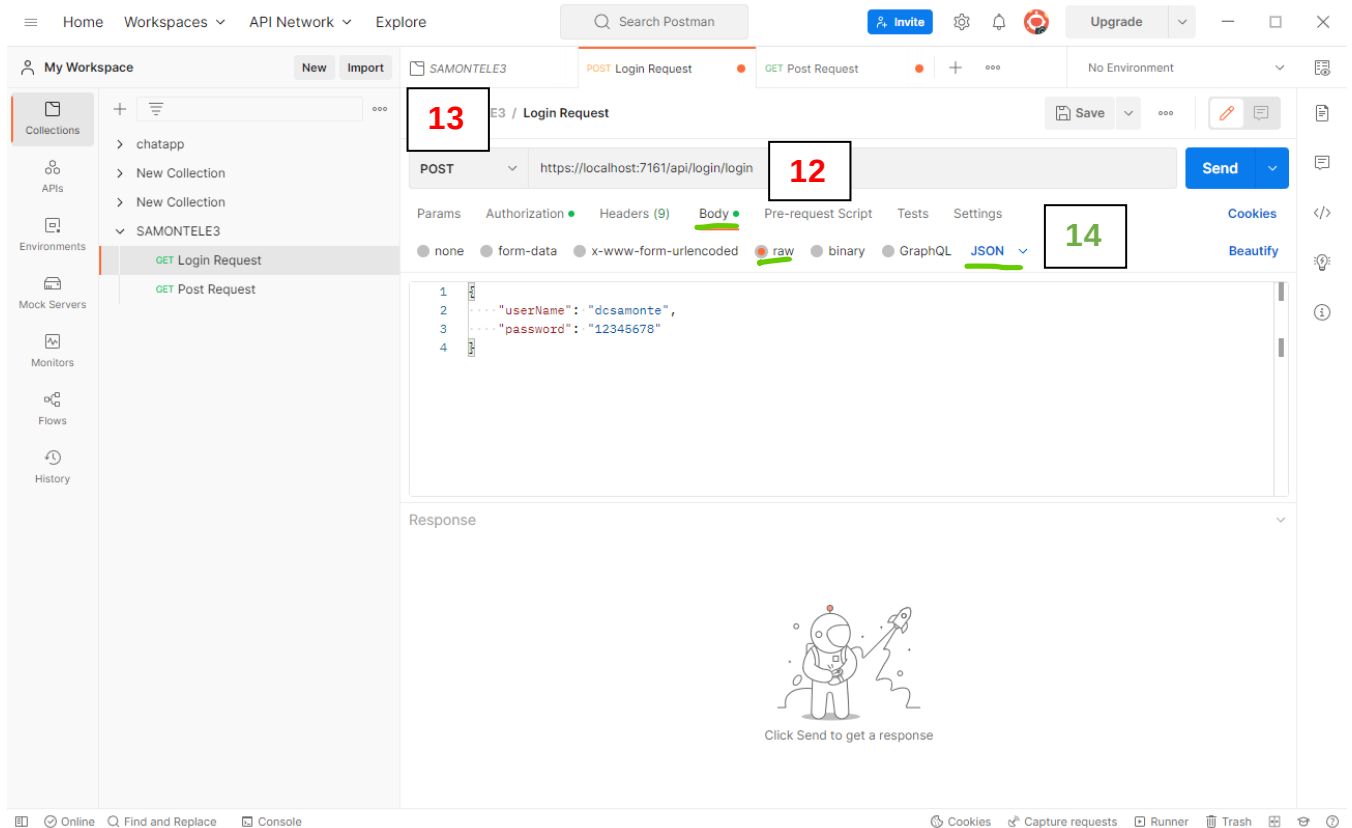
    return NotFound("User not found");
}
```

AllowAnonymous does not require authentication.

HttpPost indicates that the method for this particular route will be POST.

Route assigns the URL for this endpoint.

9. Run the project.
10. Open Postman and create a new collection, name it as <LASTNAMELE3>
11. Right click the collection and add a new request called Login Request
12. Input the URL in the textbox, following the URL schema for the login.
13. Ensure that the method is POST.
14. Go to Body, choose raw, and in the dropdown choose JSON.
15. Create a JSON body of the login details, which are username and password.



16. Hit send and there should be a token in the Response pane. Copy the string. Screenshot the Postman window.
17. In Authorization tab, in the Type dropdown, choose Bearer Token and paste the string from the login response.
18. Repeat this process whenever authorization is needed for the feature.

App feature: Register

1. In Login Controller, add a new method Register.
2. Add the following code for it:

```
[AllowAnonymous]
[HttpPost]
[Route("/api/[controller]/register")]
public ActionResult Register([FromBody] UserModel user)
{
    _db.Register(user.UserName, user.FirstName, user.LastName, user.Password);
    return Ok("User registered.");
}
```

3. In Postman, add a new request called Register Request.
4. Input the URL in the textbox, following the URL schema for the registration.
5. Ensure that the method is POST.
6. Go to Body, choose raw, and in the dropdown choose JSON.
7. Create a JSON body of the register details, which are username, last name, first name, and password.
For username and password, indicate your full name (given name, middle name, last name) with no spaces.
8. Hit Send and screenshot the result.

App functionality: List Posts

1. Add a new controller named PostController. Choose an empty API controller.
2. Add the following field and constructor:

```
private ISqlData _db;

public PostController(ISqlData db)
{
    _db = db;
}
```

3. Add a new method called ListPosts(). Type in the following code for the functionality:

```
[AllowAnonymous]
[HttpGet]
public ActionResult ListPosts()
{
    List<ListPostModel> posts = _db.ListPosts();

    return Ok(posts);
}
```

4. In Postman, add a new request called List Post Request.
5. Input the URL in the textbox, following the URL schema for listing the posts.
6. Ensure that the method is GET.
7. Hit send, and screenshot the result of the request.

App functionality: Show Post Details

1. In PostController, add a new method called ShowPostDetails().
2. Add the following code under the new method:



```
[AllowAnonymous]
[HttpGet]
[Route("/api/[controller]/{id}")]
public ActionResult ShowPostDetails(int id)
{
    ListPostModel post = _db.ShowPostDetails(id);

    return Ok(post);
}
```

3. In Postman, add a new request called Post Details Request.
4. Input the URL in the textbox, following the URL schema for showing the details of the post. Request for the post with an id of 1.
5. Ensure that the method is GET.
6. Hit send, and screenshot the result of the request.

App functionality: Add Post

1. In BlogDataLibrary, add a new class called PostForm.
2. Add the two properties of PostForm in the code:


```
public class PostForm
{
    public string Title { get; set; }
    public string Body { get; set; }
}
```

3. In Post Controller, add a new method called `GetCurrentUserId()`.
4. Add the following code under the method:

```
private int GetCurrentUserId()
{
    ClaimsIdentity identity = HttpContext.User.Identity as ClaimsIdentity;

    if (identity != null)
    {
        var userClaims = identity.Claims;
        string id = userClaims.FirstOrDefault(x => x.Type ==
ClaimTypes.NameIdentifier)?.Value;

        if (id != null)
        {
            return Convert.ToInt32(id);
        }
    }
    return 0;
}
```

5. Add the API endpoint for `AddPost()`.



```
[Authorize]
[HttpPost]
[Route("/api/[controller]/add")]
public ActionResult AddPost([FromBody] PostForm form)
{
    PostModel post = new PostModel();
    post.Title = form.Title;
    post.Body = form.Body;
    post.DateCreated = DateTime.Now;
    post.UserId = GetCurrentUser();
    _db.AddPost(post);

    return Ok("Post created.");
}
```

6. In Postman, add a new request called Add Post Request.
7. Input the URL in the textbox, following the URL schema for the adding a post.
8. Ensure that the method is POST.
9. Go to Body, choose raw, and in the dropdown choose JSON.
10. Create a JSON body of the post details, which are title and body.
 - Title: <Lastname>, <Givenname> has posted this through Postman!
 - Body: this is a blog post coming from postman!
11. Hit send, and screenshot the output.

Post-program requirements

A.

1. Screenshot the data of the Users Table of the database. Ensure that the entire Solution Explorer is included, with the folders expanded.
2. Screenshot your Postman window with the collection expanded, showing the list of requests.
3. Copy and paste all the code in LoginController and PostController.
4. **Put all screenshots and codes of this activity in a MS Word file and submit it in the LE3-1 Blackboard link provided. Filename should be LE4-1-SURNAME**

B. Run the program.

1. Create an MP4 video demonstrating the **get and post request in postman**. **Submit the MP4 video in the LE4-VID Blackboard link provided. Filename should be LE4-VID-SURNAME.**